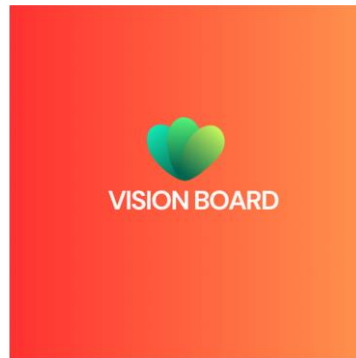




DATA ENGINEERING INTERVIEW QUESTIONS

SALARY - 40 LPA TO 50 LPA

EXPERIENCE -7 TO 15 YEARS

DEVIKRISHNA R

LinkedIn: @Devikrishna R

Email: visionboard044@gmail.com

QUESTIONS

1. What is a Slowly Changing Dimension (SCD)? Explain different types with examples.
2. Compare Star vs Snowflake schema. In what scenarios would you choose each?
3. Explain the concept of a surrogate key and its role in data warehousing.
4. What are Factless Fact Tables? When are they used?
5. How do you design a schema to handle late-arriving dimensions?

SQL for Analytics & Data Warehousing

6. Write a SQL query using a window function to calculate a rolling average over 7 days.
7. How would you handle deduplication in a massive fact table with multiple duplicates?
8. Explain how CTEs (Common Table Expressions) help in modularizing complex warehouse queries.
9. Describe a scenario where you used a CROSS JOIN effectively.
10. Write a SQL query to get the second highest sale per region using DENSE_RANK.

Performance & Optimization

11. **What is partitioning in SQL warehouses? How does it improve performance?**
12. **Difference between clustered and non-clustered indexes. Which is better in data warehousing?**
13. **How do materialized views help improve data warehouse performance?**
14. **How would you optimize a slow query running on a 10 TB fact table?**
15. **Explain the trade-offs between denormalization and normalization in warehouses.**

ETL & Data Pipeline Scenarios

16. **How do you implement CDC (Change Data Capture) in your data pipeline?**
17. **Describe your approach to incremental loading in a SQL-based data warehouse.**
18. **How do you ensure idempotency in data warehouse pipelines?**
19. **How do you validate and reconcile large volumes of data post-ingestion?**
20. **Explain your experience handling schema evolution in a data warehouse.**

Data Warehousing Concepts & Architecture

1. **What is a Slowly Changing Dimension (SCD)? Explain different types with examples.**

Answer:

SCD manages changes in dimensional data over time.

- **Type 1:** Overwrites old data (e.g., correcting a typo).
- **Type 2:** Keeps history by adding a new row (e.g., address change).
- **Type 3:** Tracks limited history in the same row (e.g., previous and current manager). Use **Type 2** in most real-world scenarios to preserve historical accuracy.

2. **Compare Star vs Snowflake schema. When would you choose each?**

Answer:

- **Star Schema:** Denormalized, faster query performance, easy joins. Ideal for dashboards.
- **Snowflake Schema:** Normalized, reduces redundancy, better for complex ETL pipelines.
Choose **Star** for speed, **Snowflake** when data size is huge or write performance matters.

3. **What is a surrogate key and why is it used?**

Answer:

A surrogate key is a system-generated unique identifier (usually an integer) used instead of natural keys.

- Ensures data consistency across systems.
- Allows tracking changes over time (e.g., in SCD Type 2).
Useful when source keys are not unique or stable.

4. What are Factless Fact Tables? When are they used?

Answer:

Factless fact tables contain no numeric measures, only foreign keys.

- Used to track **events** (e.g., student attendance) or **coverage** (e.g., promotion eligibility). Enables powerful analysis via joins and counts.
-

5. How do you handle late-arriving dimensions in a warehouse?

Answer:

- Use a **placeholder surrogate key** (e.g., -1 or "Unknown") for the missing dimension.
- Once the dimension arrives, **update** the fact record via a merge/upsert. Modern ETL tools can do this automatically with CDC and upsert logic.

SQL for Analytics & Data Warehousing

6. Write a SQL query using a window function to calculate a rolling average over 7 days.

```
SELECT
  user_id,
  event_date,
  AVG(metric_value) OVER (
    PARTITION BY user_id
    ORDER BY event_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
```

```
) AS rolling_avg_7d  
FROM events;
```

7. How do you deduplicate a large fact table with multiple duplicates?

Answer:

Use **ROW_NUMBER()** or **RANK()** with a partition.

```
WITH deduped AS (
```

```
  SELECT *,
```

```
    ROW_NUMBER() OVER (PARTITION BY transaction_id ORDER BY  
updated_at DESC) AS rn
```

```
  FROM fact_sales
```

```
)
```

```
SELECT * FROM deduped WHERE rn = 1;
```

8. Explain how CTEs help modularize complex warehouse queries.

CTEs (Common Table Expressions) break queries into **readable, reusable layers**, aiding debugging, logic reuse, and maintaining code quality.

Especially useful for **multi-step transformations**, recursive queries, and **fact-to-fact joins**.

9. Describe a scenario where you used a CROSS JOIN effectively.

Answer:

Used in **generating a calendar table**, combinations of campaigns and regions, or for **AB testing permutations**.

```
SELECT *  
FROM campaigns  
CROSS JOIN regions;
```

10. Write a SQL query to get the second highest sale per region using DENSE_RANK.

```
SELECT * FROM (  
    SELECT region, sale_amount,  
           DENSE_RANK() OVER (PARTITION BY region ORDER BY  
sale_amount DESC) AS rnk  
    FROM sales  
    ) ranked  
WHERE rnk = 2;
```

◆ Performance & Optimization

11. What is partitioning? How does it improve performance?

Answer:

Partitioning divides large tables into smaller, more manageable parts (by date, region, etc.).

- Speeds up **query performance** via partition pruning.
- Makes **ETL jobs faster** and easier to maintain.
Avoids scanning the entire table for filtered queries.

12. Difference between clustered and non-clustered indexes.

Answer:

- **Clustered Index:** Alters physical order of table. Only one allowed.
- **Non-Clustered Index:** Separate structure; multiple allowed. Use clustered on frequently queried column, non-clustered for filters and joins.

13. **How do materialized views help in warehouses?**

Answer:

Materialized views **pre-compute and store** results of heavy queries.

- Boost performance for repetitive queries.
- Scheduled refreshes maintain accuracy.
Ideal for aggregated dashboards or complex joins.

14. **How would you optimize a slow query on a 10TB fact table?**

Answer:

- Use **partitioning** and **indexing**.
- Filter early, avoid SELECT *, use projections.
- Use **CTEs or materialized views**.
- Apply **columnar formats (like Parquet)**.
- Analyze query plan and tune joins.

15. **Denormalization vs normalization: trade-offs?**

Answer:

- **Denormalization:** Faster reads, redundancy risk, storage-heavy.

- **Normalization:** Efficient storage, slower queries, complex joins. In warehousing, **denormalization preferred** for read-heavy OLAP use cases.

◆ ETL & Data Pipeline Scenarios

16. How do you implement Change Data Capture (CDC)?

Answer:

CDC tracks changes in source tables using:

- Timestamps (updated_at column)
- Triggers/log-based CDC (e.g., Debezium, SQL Server CDC)
- Merge or UPSERT operations in target warehouse

17. How do you handle incremental loading?

Answer:

- Filter data using WHERE updated_at > last_sync_time.
- Use MERGE or UPSERT to avoid duplicates.
- Maintain a **watermark table** to track last load timestamp.

18. How do you ensure idempotency in pipelines?

Answer:

- Use unique constraints + MERGE statements
- Deduplicate using ROW_NUMBER()
- Write to staging first, validate, then load final tables
Ensures reruns don't cause data duplication.

19. **How do you reconcile large volumes of data post-ingestion?**

Answer:

- Row count validation
- Checksum or hash comparison
- Summary aggregates (min/max/sum)
- Automate using **data quality frameworks** (e.g., Great Expectations)

20. **How do you handle schema evolution in a data warehouse?**

Answer:

- Use tools that support **schema-on-read** (BigQuery, Delta Lake)
- Maintain **versioned schemas**
- Use **nullable fields** or **JSON columns** for flexibility
- Backfill missing data when new columns are added